

Streamlit-Based Phishing Detection Interface

1. Introduction

Phishing email detection has developed from lexical and rule based filtering toward statistical learning and contextual language models. Yet classification performance alone does not determine whether a security tool can support meaningful decisions: predictions must be accessible, interpretable and communicated in a manner that helps users recognize uncertainty. In this setting, a user interface is not merely a visual wrapper. It mediates the interaction between model evidence, confidence, explanation and the actions a user may take after receiving a warning (Amershi et al. 2019; Abdul et al. 2018).

This review evaluates Streamlit as a framework for presenting phishing email analysis in an interactive prototype. The proposed application accepts or displays email content, invokes the underlying prediction workflow, and presents a Safe/Phishing verdict, confidence related outputs, phishing indicators and SHAP based local explanations. The research question is not whether Streamlit improves the model's discriminative ability; rather, it is how an interactive interface can expose model behavior responsibly while maintaining practical integration with a computationally demanding transformer based backend.

The review critically synthesizes human AI interaction, explainable AI (XAI), uncertainty communication, phishing warning design and interactive ML system engineering. It compares interface approaches, identifies unresolved issues and derives design and evaluation implications for the prototype.

2. Background and key concepts

2.1 Streamlit and interactive ML applications

Streamlit is a Python application framework that allows developers to create interactive data oriented web interfaces with relatively little front end code (Streamlit 2026). Widgets, session state and dynamic result presentation are useful for prototyping model demonstrations. Its script rerun execution model, however, introduces engineering considerations: expensive inference should not be triggered unintentionally by interface interactions; cached resources and persistent state require deliberate management; and long running explanations should have visible progress and failure handling. Official documentation describes these platform mechanisms but does not establish that any particular phishing prototype meets latency, security or usability targets.

2.2 The detection pipeline and its interface boundary

The project's underlying classifier is described as DeBERTa v3 large with a gated combination of contextual language representations and twelve engineered email features. DeBERTa introduced disentangled attention and enhanced decoding (He et al. 2021), while DeBERTaV3 employed replaced token detection and gradient disentangled embedding sharing in pretraining (He, Gao & Chen 2023). These architectural advances concern representation learning, not the quality of the resulting interface. The interface must therefore communicate model output without implying that a transformer architecture guarantees trustworthy predictions.

2.3 Prediction scores and uncertainty

A class probability or confidence score may look precise but is not necessarily calibrated. Guo et al. (2017) demonstrated that modern neural networks can be miscalibrated, making the interpretation of raw confidence a distinct empirical question. The interface should differentiate the predicted class, its displayed score and the correctness of the prediction. Without calibration testing, wording such as “model confidence” is more defensible than “probability the email is actually phishing.”

2.4 Local explanations

SHAP assigns feature contributions relative to a reference expectation, using a Shapley value inspired additive explanation framework (Lundberg & Lee 2017). A local explanation can organize contributions into factors supporting and opposing the explained output, with strongest factors commonly ranked by absolute contribution. The sign must be interpreted relative to the class and output space actually explained. SHAP values are not probabilities, and their causal interpretation is not justified merely by a graphical presentation (Molnar 2022).

2.5 Human centred warnings

Phishing warnings operate under conditions of limited attention, varying expertise and potential warning fatigue. Egelman, Cranor and Hong (2008) found that warning design affects user responses to phishing risks. Accordingly, a phishing analysis interface should support comprehension and action rather than presenting dense numerical outputs without context.

3. Review of existing research

3.1 Human AI interaction guidelines

Amershi et al. (2019) synthesized guidelines for designing human–AI interactions, including communicating capabilities, supporting correction and enabling appropriate reliance. For a phishing detector, this suggests clearly stating what the system analyses, providing feedback during processing, and avoiding claims of certainty when the model may err. Their work offers design principles rather than an evaluation of Streamlit specifically; its relevance is conceptual and must be translated into testable interface requirements.

3.2 Explainable interfaces and user needs

Abdul et al. (2018) surveyed human centered XAI and identified the importance of explanation purposes and user contexts. Explanation design cannot be reduced to choosing an algorithm: users may need to know why an email was flagged, what information was used, and how to respond. Kaur et al. (2020) observed that data scientists using interpretability tools can misread or over trust explanations. Although that study concerns expert users rather than email recipients, it cautions against assuming that SHAP charts are self-explanatory or automatically improve decision quality.

3.3 Explanation faithfulness and presentation

Lundberg and Lee (2017) provide a theoretical basis for additive feature attribution; however, a visually persuasive explanation is not necessarily stable or faithful under changes in background distribution, feature dependence or perturbation assumptions. Adebayo et al. (2018) demonstrated the value of sanity checks for saliency explanations, while Slack et al. (2020) showed that post hoc explanation methods can be manipulated under adversarial conditions. These findings motivate methodological validation of displayed explanations, not only attractive formatting.

3.4 Trust and overreliance

Research on explanations and decision support challenges the simplistic assumption that greater explanation detail invariably improves user outcomes. Buçinca, Malaya and Gajos (2021) showed that cognitive forcing functions can reduce overreliance on AI assistance in certain decision tasks. For phishing interfaces, this suggests that the design should encourage users to inspect email context rather than treating the model verdict as a substitute for judgement. The applicability of a specific forcing mechanism would require dedicated usability testing.

3.5 Security warning effectiveness

Egelman, Cranor and Hong (2008) demonstrated that phishing warning design influences whether users notice and act on warnings. This supports visually distinguishable Safe and Phishing outcomes and concise explanations. Nevertheless, website warning studies do not directly prove equivalent effects for a Streamlit email analysis dashboard, so extrapolation should remain cautious.

3.6 Practical framework literature

Streamlit's documentation establishes mechanisms for widgets, session state, caching and application deployment (Streamlit 2026). These are useful implementation sources but are not peer reviewed evidence of superior security, accessibility or user comprehension. A rigorous literature review must distinguish framework documentation from controlled comparative studies.

4. Comparison of existing methods or solutions

The interface decision can be assessed across rapid prototyping, control of presentation, backend integration, explainability support and operational governance. A notebook can expose raw outputs but is less suitable for nontechnical users; a Python dashboard can support rapid experimentation; a custom web frontend affords fine grained interaction control but increases implementation overhead; and a browser extension places analysis within an email reading workflow while introducing permissions and content extraction constraints. No option is universally superior because suitability depends on users, testing scope and deployment conditions.

Approach	Potential advantage	Main limitation
Jupyter notebook	Excellent for experiments; minimal deployment setup	Weak nontechnical workflow; code centric presentation
Streamlit dashboard	Rapid Python UI; interactive email entry and SHAP display	Reruns, session management and UI flexibility require care
Custom web application	Fine grained UX, accessibility and workflow control	Greater front end engineering and maintenance
Browser extension	Analysis close to email reading context	Permissions, DOM changes and extraction fragility

The comparison is an analytic synthesis of platform capabilities and HCI requirements; it is not a benchmark based on controlled experiments. Streamlit is appropriate for the project's technical demonstration, whereas the extension addresses a distinct in context interaction need.

5. Research gaps and limitations

5.1 Limited evidence for Streamlit specific phishing usability

General HCI and XAI research informs interface design, but it does not establish that a particular Streamlit dashboard helps users identify phishing accurately. Usability outcomes such as task completion time, interpretation accuracy, perceived clarity and appropriate reliance require empirical measurement.

5.2 Explanation comprehension versus visual availability

Displaying “Factors Supporting the Prediction”, “Factors Working Against the Prediction” and “Strongest SHAP Factors” does not guarantee that users understand the sign, baseline or magnitude of contributions. An opposing factor may be mistaken for a contradictory verdict, while the strongest factor may be confused with a causal explanation. User studies should assess whether labels and accompanying prose prevent these misunderstandings (Kaur et al. 2020; Molnar 2022).

5.3 Calibration and class dependent interpretation

A high model score is not proof of correctness. If the interface reports class probabilities, calibration should be evaluated using appropriate metrics and reliability diagrams; otherwise, avoid interpreting them as empirically verified risk likelihoods (Guo et al. 2017). The explained SHAP class and score space must also be documented.

5.4 Computational responsiveness

Transformer inference and repeated SHAP evaluations may produce variable latency. Streamlit’s rerun model can compound unnecessary work if execution boundaries are poorly controlled. The literature and documentation motivate caching and asynchronous feedback strategies, but actual responsiveness remains an implementation specific measurement.

5.5 Privacy and sensitive email content

Email bodies can contain personal information, confidential correspondence and links. An interface that transmits content to an API creates data handling responsibilities involving transport security, retention, logs, access controls and error messages. The existence of a local or controlled demonstration does not establish production grade privacy compliance.

5.6 Robustness and accessibility

Unusual encodings, empty fields, long messages, misleading formatting and out of distribution content may affect the analysis workflow. Color only verdicts, small SHAP labels and complex charts can exclude users or reduce comprehension. Accessibility and resilience should be explicitly tested rather than inferred from a functioning demo.

6. Implications for the proposed project

6.1 Separate model computation from presentation

The Streamlit layer should be treated as a presentation and orchestration client for the classification and explanation pipeline. The FastAPI boundary can support reuse by both Streamlit and the browser extension, provided the same preprocessing, model version and explanation semantics are applied. The presence of this architecture in the project description is not, by itself, proof of endpoint equivalence; this should be checked through integration tests.

6.2 Present a structured result

A defensible interface sequence is: extracted or entered email fields; Safe/Phishing classification; displayed score with cautious wording; observed email indicators; then a clearly labelled SHAP explanation. Separating descriptive indicators from model attributions avoids implying that every displayed indicator was necessarily a direct cause of the classification.

6.3 Explain SHAP relative to the actual target

“Supporting” and “opposing” factors should be defined in relation to the output class being explained, not automatically as phishing positive or phishing negative. “Strongest” should be described as largest absolute contribution only if the code sorts by absolute SHAP value. Where the hybrid architecture includes contextual and engineered features, the interface must not imply comprehensive attribution across both branches unless the SHAP explainer truly covers them.

6.4 Design for uncertainty and errors

A clear result should be accompanied by the recognition that false positives and false negatives remain possible. Empty or malformed emails, failed backend requests and explanation timeouts should produce informative error states rather than a misleading Safe verdict. Status messages should distinguish “analyzing”, “prediction returned” and “explanation unavailable”.

6.5 Establish an evaluation plan

Functional tests should verify field extraction, API request/response compatibility, label consistency and rendering of explanation factors. Quantitative checks should record prediction latency and explanation latency separately and evaluate score calibration if confidence is presented probabilistically. A small user study could ask participants to interpret supporting and opposing factors and choose an appropriate response to example emails, comparing the interface with and without explanations.

6.6 Define the evidence boundary

The current prototype can be described as a working interactive demonstration only to the extent supported by recorded tests. Claims of improved user trust, superior phishing detection, deployment readiness or reduced phishing susceptibility require comparative experiments. This separation strengthens academic validity by linking each claim to the appropriate form of evidence.

7. Conclusion

The literature supports Streamlit as a pragmatic environment for prototyping interactive machine learning workflows, while human–AI interaction and XAI research shows that the usefulness of an interface depends on more than rendering model output. For phishing email analysis, the central challenge is to translate a complex classifier and local feature attributions into understandable, appropriately qualified information without encouraging overconfidence.

Compared with notebooks, a Streamlit dashboard can provide a more coherent nontechnical workflow; compared with a custom frontend, it can reduce initial engineering effort; and compared with an embedded browser extension, it can provide a controlled environment for detailed inspection. These are design tradeoffs, not empirically established superiority claims. For the proposed project, the strongest rationale is to use Streamlit as an accessible validation and demonstration interface for the shared DeBERTa v3 large-gated hybrid and SHAP pipeline, then

evaluate comprehension, calibration, reliability, latency and privacy as separate research and engineering questions.

References

- Abdul, A, Vermeulen, J, Wang, D, Lim, BY & Kankanhalli, M 2018, 'Trends and trajectories for explainable, accountable and intelligible systems: An HCI research agenda', Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems, ACM, pp. 1–18, doi:10.1145/3173574.3174156
- Adebayo, J, Gilmer, J, Muelly, M, Goodfellow, I, Hardt, M & Kim, B 2018, 'Sanity checks for saliency maps', Advances in Neural Information Processing Systems, vol. 31.
- Amershi, S, Weld, D, Vorvoreanu, M, Fourney, A, Nushi, B, Collisson, P, Suh, J, Iqbal, S, Bennett, PN, Inkpen, K, Teevan, J, Kikin Gil, R & Horvitz, E 2019, 'Guidelines for human AI interaction', Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems, ACM, pp. 1–13, doi:10.1145/3290605.3300233.
- Buçinca, Z, Malaya, MB & Gajos, KZ 2021, 'To trust or to think: Cognitive forcing functions can reduce overreliance on AI in AI assisted decision making', Proceedings of the ACM on Human Computer Interaction, vol. 5, no. CSCW1, article 188, doi:10.1145/3449287.
- Egelman, S, Cranor, LF & Hong, J 2008, 'You've been warned: An empirical study of the effectiveness of web browser phishing warnings', Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, ACM, pp. 1065–1074, doi:10.1145/1357054.1357219.
- Guo, C, Pleiss, G, Sun, Y & Weinberger, KQ 2017, 'On calibration of modern neural networks', Proceedings of the 34th International Conference on Machine Learning, PMLR, vol. 70, pp. 1321–1330.
- He, P, Liu, X, Gao, J & Chen, W 2021, 'DeBERTa: Decoding enhanced BERT with disentangled attention', International Conference on Learning Representations.
- He, P, Gao, J & Chen, W 2023, 'DeBERTaV3: Improving DeBERTa using ELECTRA style pre-training with gradient disentangled embedding sharing', International Conference on Learning Representations.
- Kaur, H, Nori, H, Jenkins, S, Caruana, R, Wallach, H & Wortman Vaughan, J 2020, 'Interpreting interpretability: Understanding data scientists' use of interpretability tools for machine learning', Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems, ACM, pp. 1–14, doi:10.1145/3313831.3376219.
- Lundberg, SM & Lee, S I 2017, 'A unified approach to interpreting model predictions', Advances in Neural Information Processing Systems, vol. 30.
- Molnar, C 2022, Interpretable machine learning: A guide for making black box models explainable, 2nd edn, <https://christophm.github.io/interpretable-ml-book/>
- Slack, D, Hilgard, S, Jia, E, Singh, S & Lakkaraju, H 2020, 'Fooling LIME and SHAP: Adversarial attacks on post hoc explanation methods', Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society, ACM, pp. 180–186, doi:10.1145/3375627.3375830.
- Streamlit 2026, Streamlit documentation, Streamlit, <https://docs.streamlit.io/>